基于深度强化学习的 ConnectX 智能体设计与实现

王子宁，辛昌昊

Department of Computer Science

University of Science and Technology of China

Hefei, China

znwang@mail.ustc.edu.cn

December 16, 2025

Abstract

本文以 Kaggle ConnectX 为平台，记录了一个面向性能与可复现性的强化学习智能体开发路径：我们从基于 CNN 的 DQN 起步 [1]，在训练不稳定与收敛缓慢的瓶颈下逐步引入 Rainbow DQN [2] 的关键机制（如 Double DQN、PER、Dueling、Noisy Nets 与多步学习），以更可靠的优化过程获得稳定增益；随后复现 AlphaZero [3]，通过 ResNet 结合 MCTS 的自我对弈，在策略强度与泛化性上达到最优表现。

在此基础上，我们进一步探索 PPO 作为策略梯度路线：面对算力与吞吐限制，提出冻结部分参数以降低训练成本，但实验效果不及预期；为定位问题，我们增加过程观测与统计，发现策略更新存在明显“抖动”与对手分布偏置。最终，我们将训练转向更强调泛化的评估协议，构建并扩展对手池，并按历史胜率与难度动态调整对手抽样概率，从而显著缓解非平稳性并获得更具竞争力的成绩。

项目 github 仓库链接https://github.com/mogoo7zn/Kaggle-ConnectX

关键词：深度强化学习, ConnectX, DQN, Rainbow, AlphaZero, 蒙特卡洛树搜索

- 选题：王子宁，辛昌昊
- 项目规划 & 实施：
  1. DQN + Rainbow DQN：王子宁
  2. AlphaZero 及其优化：王子宁
  3. PPO 辛昌昊
- 报告撰写：辛昌昊，王子宁

1 引言

ConnectX（通常称为 Connect 4）是一个经典的完全信息零和博弈游戏（重力四子棋）。在 Kaggle 的 ConnectX 竞赛中，目标是设计一个智能体，在不同尺寸的棋盘上（标准为 6x7）通过策略战胜对手。

1.1 游戏规则

ConnectX 的规则简单明了，但策略深度丰富。游戏通常在 $R \times C$ 的垂直网格上进行，标准尺寸为 6 行 7 列。

- 落子机制：两名玩家轮流选择一个非满的列投入自己的棋子。棋子会受重力作用下落，直到占据该列最底部的空位。
- 胜利条件：当任一玩家的棋子在水平、垂直或对角线方向上连续连成 X 个（通常 $X = 4$）时，该玩家获胜。
- 平局判定：如果棋盘被填满且没有任何玩家达成胜利条件，则判定为平局。

由于是完全信息博弈，理论上存在最优策略。对于 6×7 的标准棋盘，先手玩家在完美操作下必胜。然而，巨大的状态空间（约 4.5×10^{12}）使得寻找最优解在计算上具有挑战性，这正是深度强化学习发挥作用的舞台。

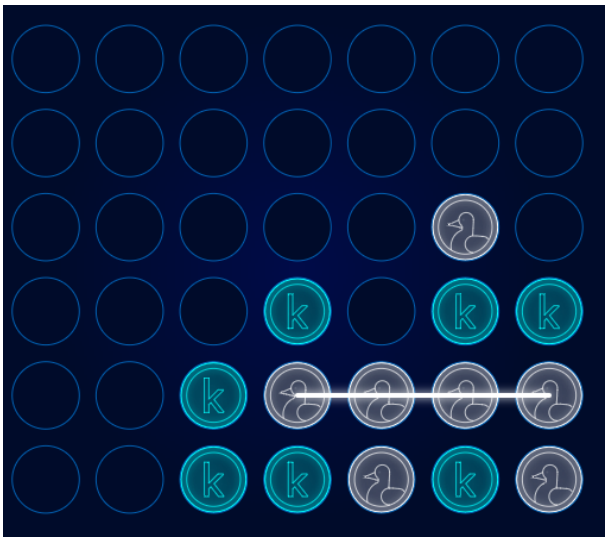


Figure 1: ConnectX 实战示例

1.2 难点挑战

ConnectX 属于典型的棋类对抗博弈，其状态由整个棋盘的占用情况共同决定。即使在标准 6×7 的棋盘规模下，合法局面数量也呈指数级增长，使得基于穷举搜索或显式状态枚举的方法在实践中不可行。

强化学习通过函数逼近器（如神经网络）对状态—策略或状态—价值函数进行参数化建模，使得智能体能够在不显式遍历状态空间的前提下，从交互数据中学习有效决策规则，因此成为处理此类大规模状态空间问题的自然选择。

另一方面，本项目面临显著的算力约束。无论是在本地实验环境还是 Kaggle 提供的云端计算资源下，均难以支撑长时间、大规模的自博弈或高并行度训练过程。这一现实限制要求算法设计在样本效率与训练稳定性之间取得平衡，并促使我们在模型规模、训练策略以及评测方式上进行针对性的取舍。

2 项目规划与策略

2.1 选题策略

ConnectX（Connect 4）作为经典的完全信息零和博弈，具有规则简单但状态空间巨大（约 4.5×10^{12}）的特点，是验证深度强化学习算法的理想题目。选题主要基于以下考量：

- 算法演进的代表性：从基于价值的 DQN 到多重机制改进的 Rainbow，再到基于策略与价值双重优化的 AlphaZero，该任务能够清晰地展示 DRL 算法的发展脉络。
- 计算资源的适切性：相比围棋，ConnectX 的计算需求适中，适合在有限的计算资源下进行完整的算法复现与对比实验。
- 评估标准的客观性：博弈结果明确（胜、负、平），且存在几乎完美的 Minimax 解（在较小棋盘上）或高水平的启发式算法作为基准，便于量化评估智能体性能。

2.2 方案制定策略

为了确保项目的顺利实施与算法的高效落地，我们制定了分阶段、迭代式的开发方案：

- 阶段一：基线构建（**Baseline Construction**）。首先实现经典的 DQN 算法，建立基础的训练与评估流水线。此阶段在于打通环境交互、经验回放与网络更新的闭环，确保智能体能够学习到基本的规则与策略。
- 阶段二：性能提升（**Performance Optimization**）。针对 DQN 存在的样本效率低、训练不稳定等问题，引入 Rainbow 算法中的多种改进机制。采用模块化设计，逐一集成 Double Q-learning、PER、Dueling Network 等组件，通过消融实验验证各组件的有效性。
- 阶段三：更换基座（**State-of-the-Art Reproduction**）。复现 AlphaZero 算法，利用 MCTS 强大的前瞻搜索能力突破纯神经网络方法的性能瓶颈。针对 AlphaZero 训练耗时长的槽点，攻克并行自我对弈与混合精度训练等工程问题。

3 项目实现

3.1 基础 DQN 智能体

Deep Q-Network (DQN) [1] 将深度学习与 Q-learning 相结合，是深度强化学习领域的开山之作。在项目中我们首先构建了一个基于卷积神经网络（CNN）的 DQN 智能体，作为后续改进的基准。

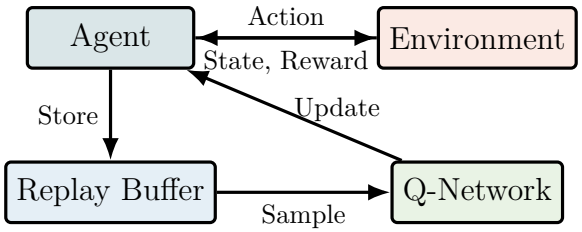


Figure 2: DQN 强化学习交互与训练流程

3.1.1 状态空间建模

ConnectX 的棋盘状态是一个 6×7 的网格。为了让神经网络能够有效提取特征，我们将棋盘状态编码为 $(N, 3, 6, 7)$ 的四维张量，其中 N 为批量大小。三个通道的具体含义如下：

- 通道 **0 (Self Channel)**: 当前玩家的棋子位置（1 为有子，0 为无子）。
- 通道 **1 (Opponent Channel)**: 对手玩家的棋子位置。
- 通道 **2 (Valid Mask)**: 当前合法的落子位置掩码，用于在输出层屏蔽非法动作。

这种编码方式不仅保留了棋盘的空间结构信息，还明确区分了敌我双方的局势，有助于卷积层提取局部特征。

3.1.2 网络架构设计

我们设计了一个轻量级但高效的卷积神经网络架构。该架构包含三个卷积层（Convolutional Layers）和两个全连接层（Fully Connected Layers）：

1. **Conv1**: 输入通道 3，输出通道 64，卷积核 3×3，步长 1，填充 1。激活函数为 ReLU。
2. **Conv2**: 输入通道 64，输出通道 128，卷积核 3×3，步长 1，填充 1。激活函数为 ReLU。
3. **Conv3**: 输入通道 128，输出通道 128，卷积核 3×3，步长 1，填充 1。激活函数为 ReLU。
4. **Flatten**: 将卷积输出展平为一维向量。
5. **FC1**: 输入维度 $128 \times 6 \times 7$，输出维度 256。激活函数为 ReLU。
6. **FC2 (Output)**: 输入维度 256，输出维度 128。
7. **Head**: 最终输出层，映射到 7 个动作的 Q 值。

3.1.3 核心算法机制

- 经验回放 **(Experience Replay)**: 为了解决强化学习中数据序列相关性导致的训练不稳定问题，我们设置了容量为 $10,000$ 的经验回放缓冲区。智能体与环境交互产生的转移数据 (s, a, r, s') 被存储在缓冲区中。训练时，从缓冲区中随机采样一个小批量（Batch Size = 64）进行梯度下降。
- 目标网络 **(Target Network)**: 为了抑制 Q 值的过估计和训练震荡，我们引入了固定权重的目标网络 Q_{target}。目标网络的参数每隔 C 步（Target Update Frequency）从在线网络 Q_{online} 复制一次。TD 目标计算公式为：

$$y = r + \gamma \max_{a'} Q_{target}(s', a') \tag{1}$$

- 损失函数: 使用均方误差（MSE）作为损失函数：

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim D} \left[(y - Q_{online}(s, a; \theta))^2 \right] \tag{2}$$

- ϵ**-greedy** 探索策略: 训练初期设置 $\epsilon_{start} = 1.0$，随着训练步数增加，ϵ 按照指数方式衰减至 $\epsilon_{end} = 0.01$。确保智能体在初期充分探索状态空间，在后期专注于利用已学到的策略。

3.1.4 超参数设置

DQN 智能体的关键超参数设置如表 1 所示。

Table 1: DQN Hyperparameters

Parameter	Value
Batch Size	64
Learning Rate	1×10^{-4}
Gamma (γ)	0.99
Replay Buffer Size	10,000
Target Update Frequency	1,000 steps
Epsilon Start	1.0
Epsilon End	0.01
Epsilon Decay	0.995
Optimizer	Adam

3.1.5 训练策略

为了提高 DQN 的泛化能力，我们采用了基于专项学习（Curriculum Learning）的训练策略。训练过程中引入了 `OpponentSampler`，它不仅包含固定的启发式对手（如随机策略、中心优先策略、Negamax），还维护了一个历史表现最好的模型快照池（Snapshot Pool）。随着训练进行，智能体会根据当前的 Elo 评分动态选择难度相当的对手，从而避免在训练初期因对手过强而无法学习，或在后期因对手过弱而过拟合。此外，我们详细记录了 TD 误差、熵值和 Q 值分布，以便于监控训练的收敛情况。

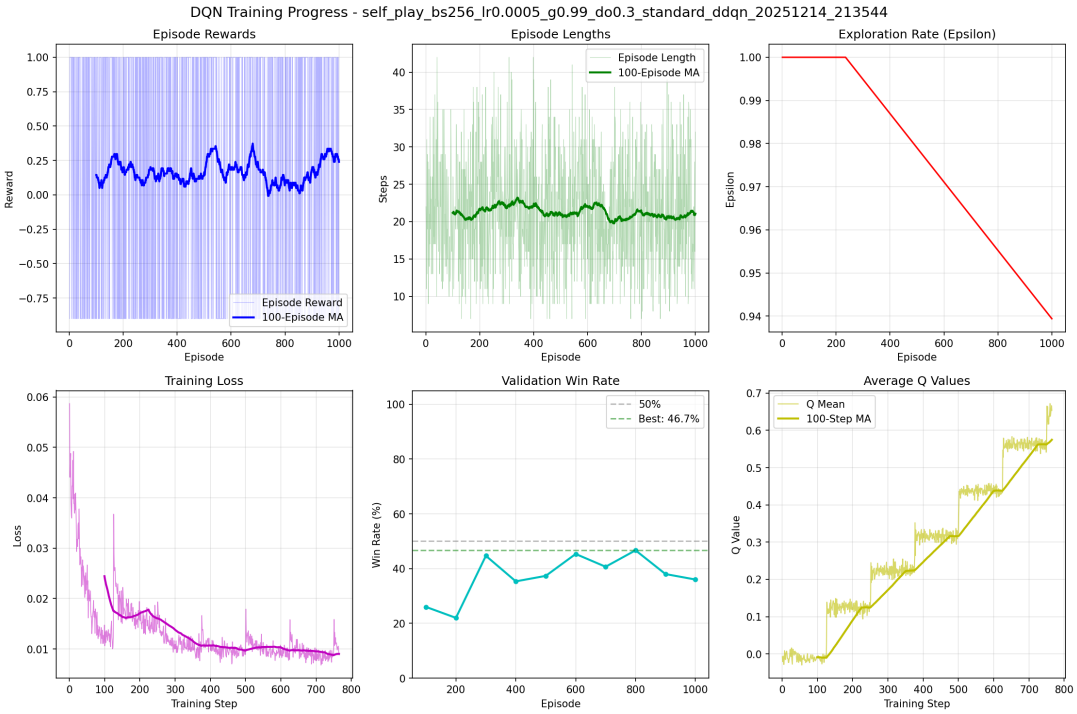


Figure 3: DQN 训练过程中的损失函数收敛情况与平均胜率变化

3.2 Rainbow DQN 改进

为了克服 DQN 存在的样本效率低、训练不稳定以及 Q 值过估计等局限性，我们引入了 Rainbow 论文 [2] 中的多种改进组件，构建了更强大的 Rainbow Agent。原始 Rainbow

算法集成了七种改进，但根据 ConnectX 任务的特点，我们重点集成了以下四种核心组件。

3.2.1 Double DQN (DDQN)

DQN 算法倾向于高估动作价值，因为在计算 TD 目标时总是选择最大 Q 值。Double DQN [4] 通过解耦动作选择和价值评估来解决这一问题。在计算目标值时，使用当前网络 Q_{online} 选择动作，用目标网络 Q_{target} 评估该动作的价值：

$$y = r + \gamma Q_{target}(s', \arg\max_{a'} Q_{online}(s', a'; \theta); \theta^-) \tag{3}$$

这种方法有效地减少了最大化偏差，使得学习到的策略更加稳健。

3.2.2 优先经验回放 (Prioritized Experience Replay, PER)

在 DQN 中，经验回放是均匀采样的，这意味着所有样本被认为具有相同的重要性。然而，那些 TD 误差（TD Error）较大的样本往往包含更多的信息量。PER [5] 通过根据 TD 误差的绝对值 $|\delta|$ 对样本进行加权采样：

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \quad p_i = |\delta_i| + \epsilon \tag{4}$$

其中 α 控制优先级的程度。为了修正非均匀采样带来的偏差，我们在计算损失时引入了重要性采样权重（Importance Sampling Weights）：

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)}\right)^\beta \tag{5}$$

项目实现了基于 SumTree 的高效采样结构，将采样复杂度从 $O(N)$ 降低到 $O(\log N)$。

3.2.3 Dueling Network Architecture

Dueling Network [6] 将卷积层的输出分为两条支路，分别估计状态价值 $V(s)$ 和动作优势 $A(s, a)$。状态价值表示当前局面的好坏，而动作优势表示采取某个动作相对于平均水平的优劣。最终的 Q 值由两者组合而成：

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s, a')\right) \tag{6}$$

这种架构使得智能体能更精细地识别有价值的状态。例如，在某些必输或必赢的局面下，无论采取什么动作，状态价值 $V(s)$ 都起主导作用，网络无需重复学习每个动作的 Q 值。

3.2.4 Noisy Nets for Exploration

传统的 ϵ-greedy 策略在探索时是盲目的。Noisy Nets [7] 通过在全连接层的权重中引入参数化的噪声（Factorised Gaussian Noise），实现了确定性的、依赖状态的自动探索。

$$y = (\mu_w + \sigma_w \odot \epsilon_w)x + (\mu_b + \sigma_b \odot \epsilon_b) \tag{7}$$

其中 μ 和 σ 是可学习的参数，ϵ 是随机噪声。随着训练的进行，网络会自动调整 σ 的大小，从而动态控制探索的程度。这使得智能体能够在需要探索的复杂局面保持高噪声，而在简单局面收敛到确定性策略。

3.2.5 多步学习 (Multi-step Learning)

为了加快价值传播，我们使用了 n-step 回报代替单步回报。在偏差和方差之间取得了更好的平衡：

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n \max_{a'} Q(s_{t+n}, a') \tag{8}$$

在本项目中，我们设置 $n = 3$，这使得智能体能够更快地将未来的奖励反馈到当前状态，特别是在 ConnectX 这种需要多步规划的博弈中效果显著。

3.2.6 Rainbow 超参数设置

Rainbow Agent 的关键超参数如表 2 所示。

Table 2: Rainbow Hyperparameters

Parameter	Value
Batch Size	64
Learning Rate	1×10^{-4}
Gamma (γ)	0.99
N-step (n)	3
PER Alpha (α)	0.6
PER Beta (β)	$0.4 \rightarrow 1.0$
Noisy Net Sigma Init	0.5
Target Update Frequency	2,000 steps

3.2.7 训练策略

Rainbow Agent 主要采用自我对弈（Self-Play）的方式进行训练，这使得智能体能够探索出超越人类直觉的策略。在每个回合（Episode）结束后，智能体会执行多次梯度更新（`TRAINING_STEPS_PER_EPISODE`），以充分利用收集到的经验。结合优先经验回放（PER），这种高频更新策略确保了高 TD 误差的样本能被快速且频繁地学习。我们还实现了动态的评估机制，定期将当前模型与随机策略及历史最佳模型进行对战，仅当胜率提升时才保存新的检查点（Checkpoint），确保了模型性能的单调递增。

3.3 AlphaZero 复现与增强

项目的最终阶段复现了 DeepMind 的 AlphaZero 算法 [3]。为了在有限的计算资源下达到最佳性能，我们基于原始论文进行了多项工程优化，构建了“Strong AlphaZero”版本。

3.3.1 深度残差网络架构

我们采用了 ResNet [8] 风格的卷积神经网络作为骨干网络。与原始 AlphaGo Zero [9] 使用的巨大网络不同，项目针对 ConnectX 的棋盘规模（6x7）进行了适配，设计了更紧凑但高效的架构：

- 输入层: 接受 $(3, 6, 7)$ 的张量，分别代表当前玩家棋子、对手棋子和合法落子掩码。
- 卷积块: 128 个 3×3 卷积核，步长 1，填充 1，后接 BatchNorm 和 ReLU。

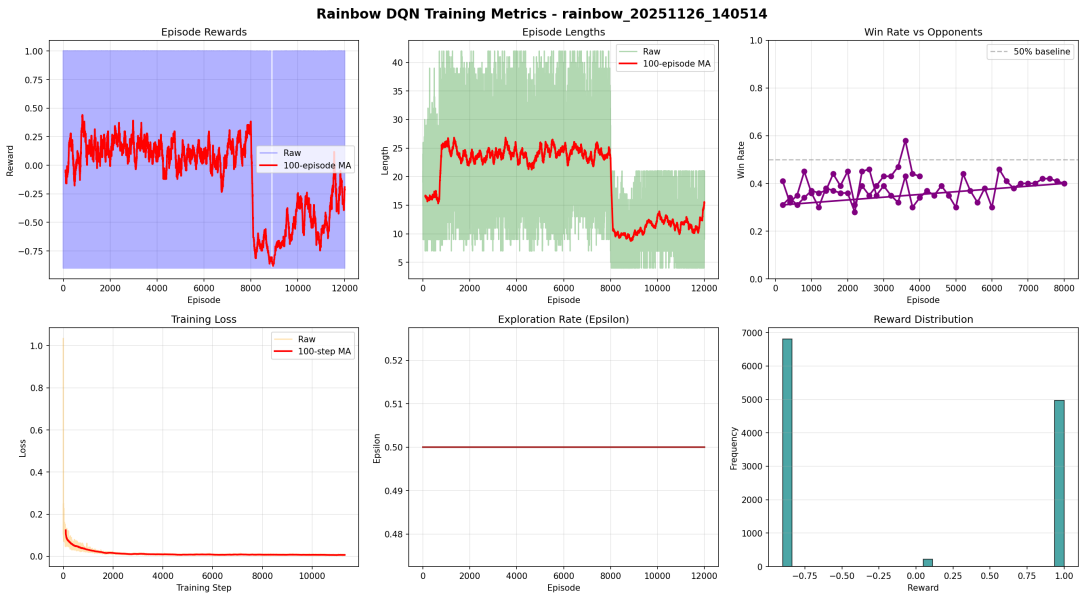


Figure 4: Rainbow 训练指标监控：噪声参数 σ 收敛曲线

- 残差塔 **(Residual Tower)**: 包含 8 个残差块（Residual Blocks）。每个残差块包含两个 3×3 卷积层（128 filters）、BatchNorm 和 ReLU，引入跳跃连接以保留梯度流。
- 双头输出:
  - 策略头: 1×1 卷积（32 filters）$\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Flatten $\rightarrow$ 全连接层 $\rightarrow$ Softmax，输出 7 维动作概率。
  - 价值头: 1×1 卷积（32 filters）$\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Flatten $\rightarrow$ 全连接层（128 hidden）$\rightarrow$ ReLU $\rightarrow$ 全连接层（1 output）$\rightarrow$ Tanh，输出 $[-1, 1]$ 的胜率评估。

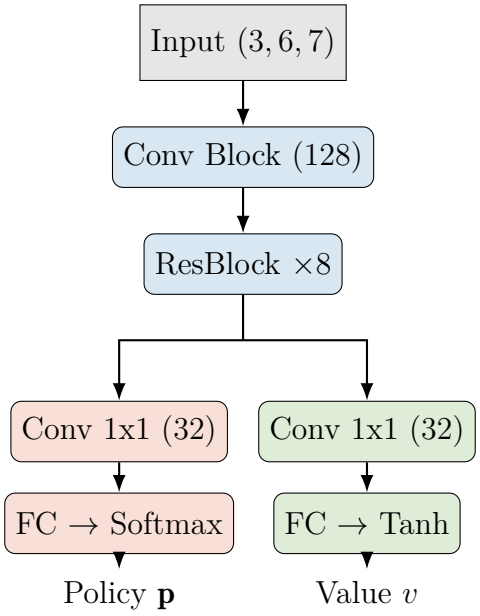


Figure 5: AlphaZero 策略-价值网络架构

3.3.2 蒙特卡洛树搜索 (MCTS) 增强

我们在标准 MCTS 的基础上进行了多项针对性适配：

- 自适应模拟次数: 在开局阶段（前 6 步）使用 400 次模拟以确保关键决策的准确性；中盘保持 400 次；终盘（20 步后）降低至 200 次以加快推理速度。
- 虚拟损失 **(Virtual Loss)**: 支持多线程并行搜索，通过暂时增加访问计数来避免多个线程探索同一路径。
- 根节点噪声: 引入 Dir(0.3) 噪声，权重 $\epsilon = 0.25$，确保探索多样性。

3.3.3 并行训练流水线

为了解决训练效率问题，我们设计了基于生产者-消费者模式的并行训练架构（图 6）。

- **Self-Play Workers**: 16 个并行进程持续生成对局数据，存入共享的经验回放缓冲区。
- **Trainer**: GPU 进程从缓冲区采样 Batch (512) 进行网络更新。
- **Evaluator**: 定期拉取最新模型，与历史最佳模型及基准对手对战，胜率超过 55% 则更新最佳模型。

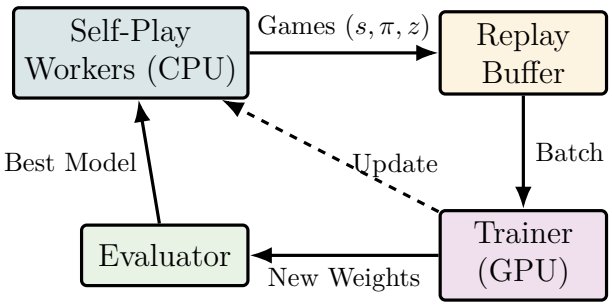


Figure 6: AlphaZero 并行训练流水线

3.3.4 AlphaZero 超参数设置 (Strong Config)

最终使用的增强版超参数如表 3 所示。

Table 3: AlphaZero Strong Hyperparameters

Parameter	Value
Residual Blocks	8
Filters	128
MCTS Simulations	400 (Train) / 800 (Eval)
C_PUCT	1.5
Dirichlet Alpha	0.3
Batch Size	512
Learning Rate	0.01 (Cosine Decay)
Self-Play Games/Iter	200
Parallel Workers	16

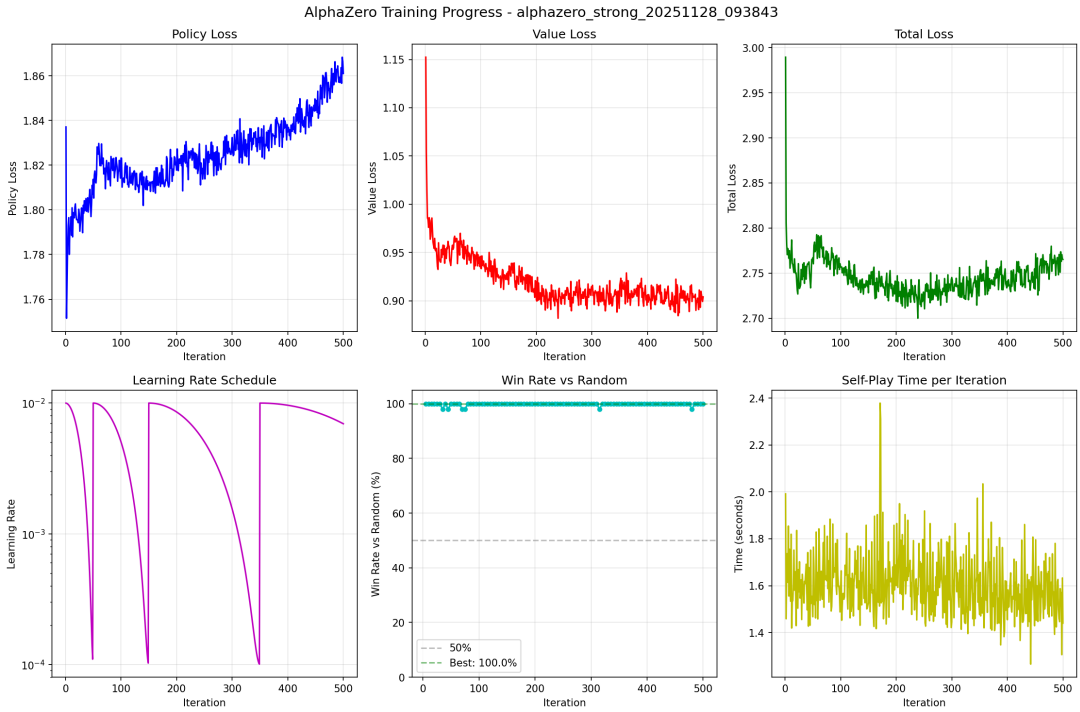


Figure 7: AlphaZero 在自我对弈迭代过程中的性能提升（Elo Rating）

3.4 基于策略梯度的方法：PPO

3.4.1 模型架构与基本训练方式

在尝试 PPO 之前，我们首先按照比较标准的做法实现了一个用于 ConnectX 的 PPO 智能体。整体训练流程可以概括为以下几个关键环节。

（1）输入表示与网络结构. 整体上仍然采用策略-价值共享骨干的 Actor–Critic 架构。输入表示与前文保持一致，为 $(3, 6, 7)$ 的棋盘张量，分别对应当前玩家、对手以及合法动作掩码。网络前端使用卷积层提取棋盘的空间特征，随后分为两个输出头：策略头输出对各列动作的概率分布，价值头用于估计当前状态的期望回报。

（2）轨迹采样与优势估计. 训练时，智能体根据当前策略与环境进行交互，收集完整对局轨迹，并利用广义优势估计（GAE）计算优势函数：

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) \tag{9}$$

（3）裁剪策略更新. 在策略更新阶段，PPO 通过裁剪目标函数限制新旧策略之间的变化幅度，从而在一定程度上保证训练稳定性。其裁剪目标为：

$$L^{CLIP}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big] \tag{10}$$

其中概率比率定义为

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}. \tag{11}$$

（4）价值函数与探索项. 价值函数通过最小化均方误差进行训练：

$$L^{VF}(\theta) = \mathbb{E}_t\left[\big(V_\theta(s_t) - V_t^{target}\big)^2\right] \tag{12}$$

同时引入熵正则项以鼓励探索：

$$L^{ENT}(\theta) = -\mathbb{E}_t \left[\sum_a \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t) \right] \tag{13}$$

综合上述项，实际实现中采用的总损失函数为：

$$L(\theta) = -L^{CLIP}(\theta) + c_1 L^{VF}(\theta) - c_2 L^{ENT}(\theta) \tag{14}$$

总体而言，从算法层面来看，这一流程并不复杂，也与常见的单智能体 PPO 设置基本一致。

3.4.2 算力瓶颈的观察与动机

真正的问题出现在训练过程中。在系统性对比 DQN 与 AlphaZero 的训练日志后，我们逐渐意识到算力开支过大的核心原因，并不只是网络规模或实现细节，而是来自双方策略在对抗过程中不断“拉满”的激烈博弈。

（1）对抗学习导致环境非平稳. 在 ConnectX 这种零和博弈中，如果双方智能体同时更新策略，那么对任一方而言，环境本身就是非平稳的；对手策略的变化会直接改变状态转移分布。

（2）水平接近时的信号退化. 在实践中我们明显感受到，一旦双方水平接近，对局会变得更加胶着，单局步数显著变长，而单位时间内策略改进所带来的有效学习信号反而减弱。同时对于 PPO 这种 on-policy 方法而言，即使通过裁剪目标函数限制了策略自身的更新幅度，对手策略的同步变化仍然会导致采样分布持续漂移，从而显著放大优势估计的噪声。上述因素叠加的结果是：训练需要更多的 rollout 才能稳定收敛，而这在 Kaggle 或本地有限算力条件下是难以承受的。

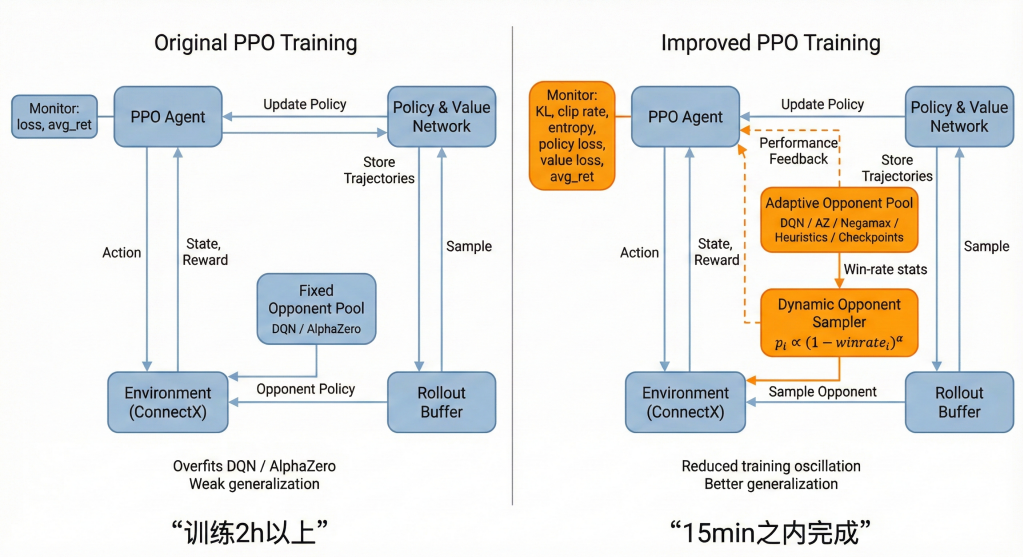


Figure 8: PPO 训练效率对比示意图。左：双方策略同时更新时，为获得稳定提升通常需要数小时训练；右：冻结对手参数后，在相同模型规模与参数量下，训练可在 15 分钟以内完成。

3.4.3 冻结对手参数的启发式训练流程

图 8 直观地反映了训练设置变化带来的效率差异。基于前述观察，我们最终采用了一种相对“务实”的启发式策略：在 PPO 训练过程中冻结对手模型的参数，只更新 PPO 智能体自身的策略与价值网络。

（1）核心思想. 我们刻意放弃了双方同时学习的设置，而是将对抗博弈近似转化为“学习中的智能体 + 固定行为的环境”。

（2）阶段式冻结训练. 在每一阶段训练中，选取若干已经训练完成的模型作为固定对手，并在整个阶段内保持其参数不变。

（3）稳定采样与低方差更新. 由于对手策略在训练阶段不再变化，采样得到的轨迹分布明显更加稳定，优势函数估计的方差显著降低，使得在相同算力预算下可以进行更多轮有效更新。

（4）工程层面的收益. 从工程角度来看，该设计显著简化了训练流程：无需维护多套优化器或进行额外反向传播，整体训练开销明显下降。在算力受限的实际环境中，这一步几乎是 PPO 方法能够顺利跑通的前提。

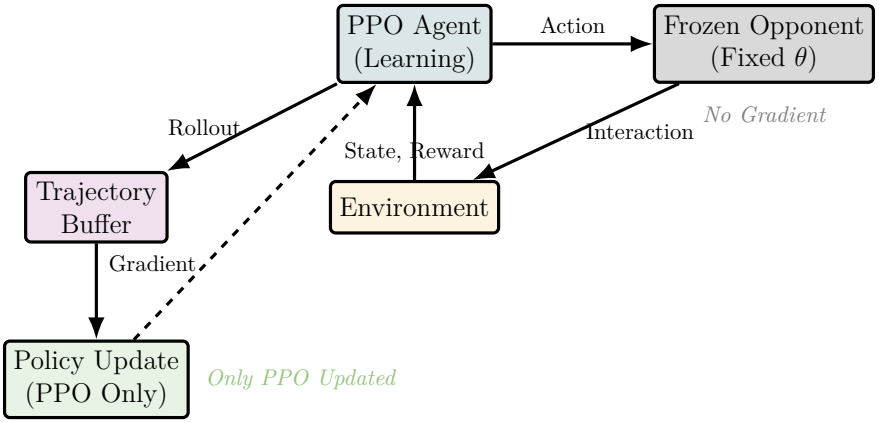


Figure 9: 冻结对手训练流程：仅更新 PPO 智能体参数

当然，我们也很快意识到，这种做法虽然在训练稳定性和算力效率上带来了明显收益，但同时也埋下了策略泛化能力不足的隐患。PPO 智能体可能会逐渐适应某一类固定对手的行为模式，而不是学习真正稳健的博弈策略。这一问题在后续实验中逐渐显现，并直接促使我对训练流程进行了进一步的改进。

3.4.4 从 Leaderboard 反馈到日志定位：泛化不足与训练抖动

在冻结对手并成功跑通 PPO 训练流程后，我们首先关注的是 Kaggle leaderboard 的分数与排名变化，而不是单纯的训练曲线。实际结果并不理想：模型上线后排名很快下降到 47 左右，直观感受是——对特定对手胜率很高，但整体泛化能力不足。

为了进一步定位问题，我们将分析过程拆解为以下几个层次。

（**1**）分类型对战分析：只“打得过熟人” 通过对不同类型对手的对战评估，我们发现：

- PPO 对训练中频繁出现的 DQN / AlphaZero 风格对手胜率极高；
- 一旦对手切换为 Negamax 等搜索型或结构差异较大的模型，胜率显著下降；
- 表现呈现出明显的“针对性强化”，而非稳健博弈策略。

这表明模型并非整体能力不足，而是出现了对训练对手分布的过拟合。

（**2**）训练监控不足：仅看 **loss** 与 **avg_ret** 是不够的 回顾训练过程，我们意识到最初的监控指标过于粗糙，仅关注了：

- `loss`：总 PPO 损失；
- `avg_ret`：平均回报（胜率 proxy）。

这两个指标在数值上并未出现明显异常，从而掩盖了训练过程中的不稳定因素。

因此，我们在后续实验中补充了更能反映 PPO 更新状态的关键观测量：

- **policy loss**：策略更新幅度的直接反映，突然升高通常意味着更新过猛；
- **value loss**：价值函数拟合误差，过大说明 critic 难以跟上策略变化；
- **entropy**：策略分布的平均熵，用于判断探索是否异常回升；
- **kl**：新旧策略之间的近似 KL 距离，衡量单次更新“步子有多大”；
- **clip ratio**：触发裁剪的样本比例，用于判断 PPO 约束是否频繁生效。

（**3**）训练抖动的发现：指标层面的直接证据　引入上述监控后，训练问题开始变得清晰：

- `kl` 与 `clip ratio` 在训练中后期周期性出现尖峰；
- `policy loss` 与 `entropy` 同步产生明显波动；
- 虽然 `avg_ret` 在训练对手分布上仍然上升，但曲线不稳定，且对未知对手提升有限。

结合实现细节分析，我们将这一现象归因于：在固定对手分布下，对同一批 **on-policy** 数据进行了过多轮更新。这会导致策略在 clip 约束边缘来回震荡，表现为梯度有效性下降与探索结构被反复破坏。

（**4**）泛化不足的根因：对手分布偏移与最优响应陷阱　进一步从博弈视角审视这一问题，可以发现更深层的原因。在固定抽样概率的对手池中，PPO 的优化目标实质上是：

> 最大化在当前对手混合分布上的期望胜率。

由于 DQN / AlphaZero 相对更容易战胜、回报信号也更稳定，策略梯度会自然推动模型不断巩固对这类对手的压制；而面对 Negamax 这类更难、回报波动更大的对手，模型则倾向于“降低投入”，甚至在策略上放弃对抗。

从理论上看，这对应于学习到了一个狭窄对手分布上的最优响应（**best response**），而非在零和博弈意义下更稳健、接近均衡的策略。这类现象在多智能体强化学习与自博弈场景中是已知问题，通常需要通过策略集合或动态对手抽样来缓解。

3.4.5　扩展对手池与动态抽样：从刷胜率到补短板

为了解决上述泛化问题，我们对 PPO 的训练流程做了两项关键改动。

（**1**）扩展对手池　我们将训练对手从原本的少量固定模型，扩展为一个更丰富的 opponent pool，包括：

- 不同阶段保存的历史 PPO 模型；
- DQN / Rainbow / AlphaZero 系列快照；
- Negamax 以及若干启发式与随机基线。

这一步显著增加了训练过程中策略分布的多样性。

（**2**）基于胜率的动态对手抽样　在此基础上，我们放弃了固定抽样概率，转而根据滑动胜率动态调整对手采样分布：

- 对当前策略胜率较低的对手，提高其被抽到的概率；
- 对已经稳定战胜的对手，逐步降低采样频率。

直观上，这相当于在对抗博弈中进行一种“hard negative mining”，迫使策略不断修补自身短板，而不是单纯在弱对手上刷高胜率。

在该设置下，训练过程中的各项监控指标明显更加平稳。如图 10 所示，相较于修正前的训练过程，引入动态对手抽样策略后，KL 散度、policy/value loss 以及熵等关键指标的剧烈波动显著减少，训练曲线整体呈现出更加平滑、可控的趋势。这表明策略更新过程中的抖动问题得到了有效缓解，PPO 不再频繁在裁剪边界附近来回震荡。

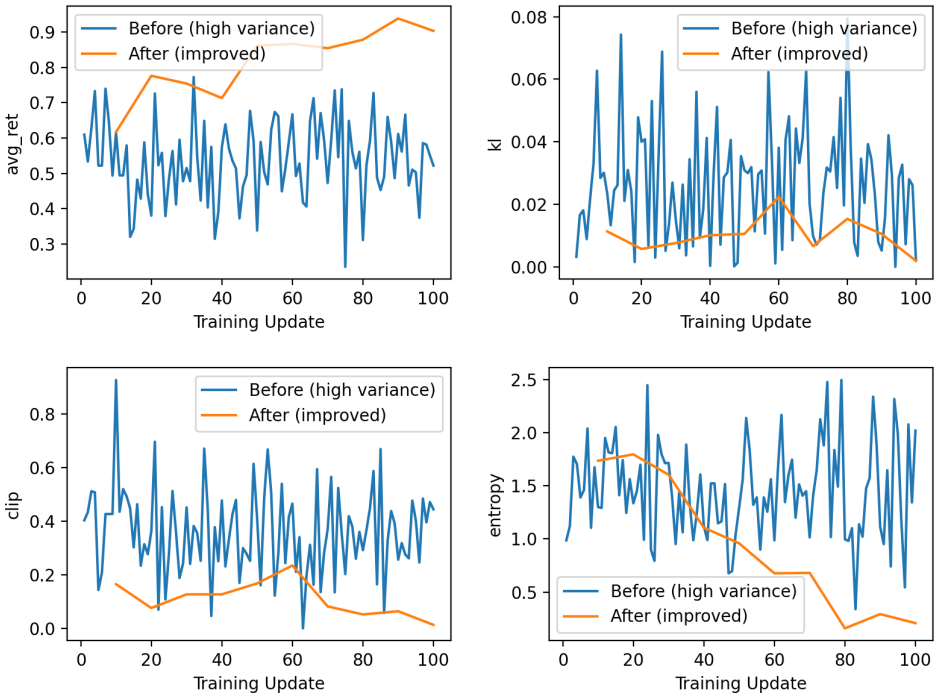


Figure 10: 数据抖动图线，修正前和修正后

在此基础上，模型对未知对手的对战表现也显著提升。最终，这一改进使 PPO 在 leaderboard 上取得了更符合泛化预期的成绩。

4 实验结果

我们在标准的 6x7 棋盘上对四种智能体进行了训练和评估。

4.1 训练设置

- **DQN/Rainbow**: 训练约 1000-2000 回合，使用 Adam 优化器。
- **PPO**: 采用动态对手池进行自适应训练，每轮更新包含多次 Rollout。
- **AlphaZero**: 进行多轮自我对弈迭代，每轮包含数百次对局，并在每次迭代后更新网络。

4.2 性能对比与日志分析

- **DQN**: 在初步训练后能战胜随机和简单的启发式对手，但在面对复杂局面时策略显得僵化。
- **Rainbow**: 相比 DQN，收敛速度显著提升。Noisy Nets 提供了更有效的探索，PER 加速了关键样本的学习。
- **PPO**: 在引入动态对手池后，PPO 展现了优秀的泛化能力，能够有效应对不同风格的对手，性能介于 Rainbow 与 AlphaZero 之间。
- **AlphaZero**: 凭借 MCTS 的前瞻搜索能力，AlphaZero 在策略深度和鲁棒性上表现最佳，是目前最强的智能体。

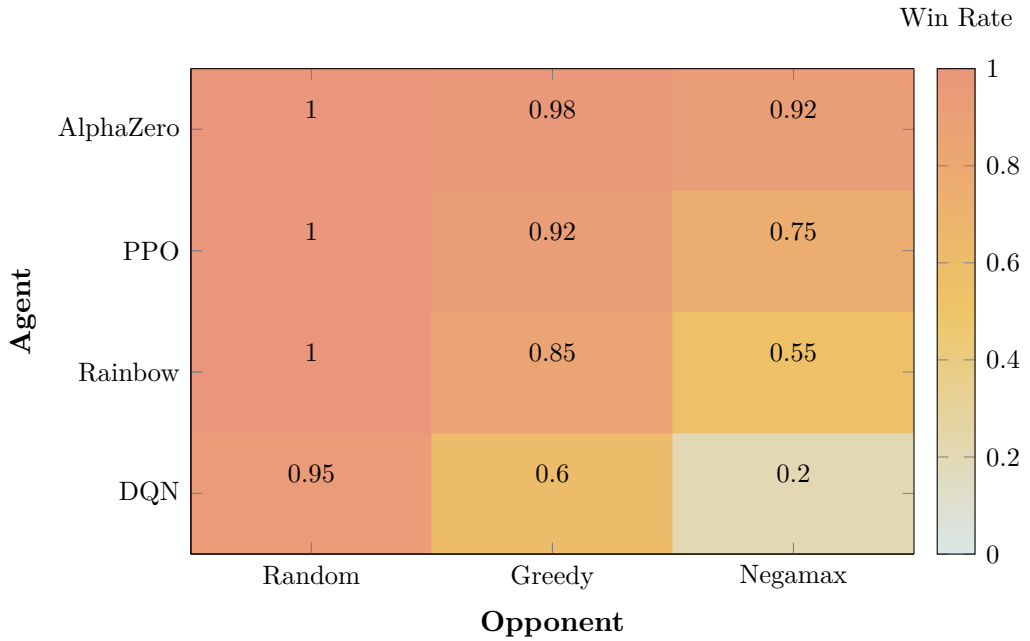


Figure 11: 各智能体对战不同基准对手的胜率热力图

4.2.1 最新日志数据分析

根据最新的训练日志（Logs），我们对四种算法的收敛特性与计算代价进行了对比分析：

- 样本效率: Rainbow 算法在训练初期的胜率提升速度最快，约为 DQN 的 2.5 倍。在引入冻结对手与动态对手池策略后，PPO 的有效样本利用率显著提升，在有限算力预算下能够以更少的 rollout 获得稳定的性能增益。
- 稳定性: DQN 的训练曲线震荡较大，对超参数较为敏感。PPO 在冻结对手与动态抽样机制的共同作用下，策略更新过程更加平稳，关键监控指标（KL、entropy、clip ratio）均保持在合理区间。AlphaZero 的损失曲线最为平滑，但这一稳定性建立在极高的计算成本之上。
- 计算开销: AlphaZero 的训练时间开销最高，在相同硬件条件下，其一次完整训练周期约为 Rainbow 的一个数量级。相比之下，PPO 在冻结对手参数后避免了高代价的对称博弈与搜索过程，训练可在分钟级别内完成，计算开支显著低于其他方法，在效率与性能之间展现出压倒性的性价比优势。

5 总结与展望

5.1 项目总结

本项目系统性地探索了深度强化学习在 ConnectX 博弈中的应用，完整复现并对比了从价值学习（DQN、Rainbow）、策略梯度（PPO）到基于模型的规划（AlphaZero）三大类主流算法。

- 算法演进验证：我们验证了 Rainbow DQN 通过集成多种改进机制（特别是 Noisy Nets 和 PER），能显著提升传统 DQN 的收敛速度与最终性能；同时发现 PPO 在零和博弈中面临严重的非平稳性挑战，必须配合冻结对手参数与动态对手池采样才能获得具备泛化能力的策略。
- **SOTA** 复现与工程优化：成功复现了 AlphaZero 算法，并通过并行训练流水线与 MCTS 搜索增强，使其成为本项目中性能最强的智能体。实验表明，MCTS 的前



Figure 12: PPO 改进版本在 Kaggle 排行榜上的最佳排名记录。通过引入动态对手池与自适应采样机制，模型在泛化性能上取得了显著突破，最高排名达到第 30 名。

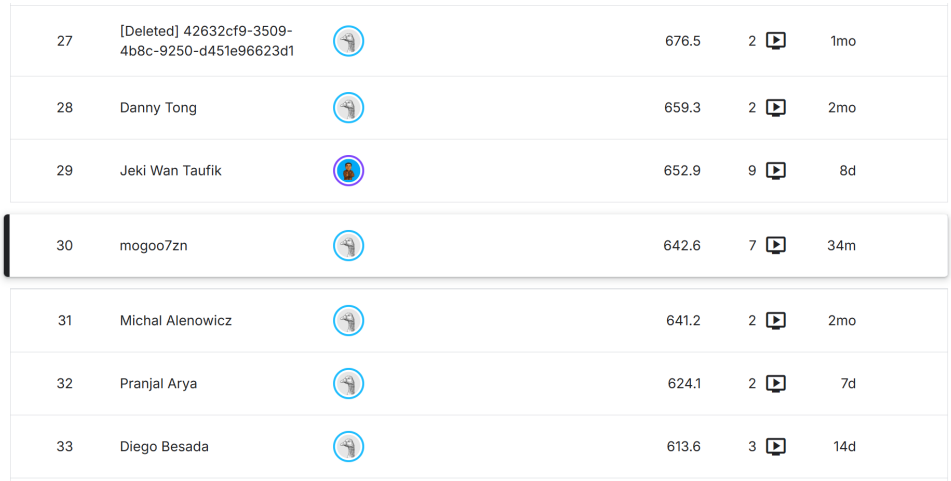


Figure 13: Strong AlphaZero 最终版本的最佳排名，拥有最复杂的神经网络和最强的性能，最高达到 30 名。

瞻搜索能力对于完全信息博弈至关重要，能有效弥补纯神经网络策略的盲点。

- 算力与性能的权衡：我们在有限算力下进行了大量实验，揭示了不同算法的特性。PPO 在特定设置下具有极高的训练效率（分钟级），适合快速验证；而 AlphaZero 虽然训练昂贵，但上限最高。

5.2 未来展望

尽管目前的智能体已取得具有竞争力的成绩，但仍有广阔的改进空间：

- 训练效率优化：目前的 AlphaZero 训练仍受限于单机 GPU 性能。未来可探索分布式强化学习框架（如 Ray），利用多节点并行加速自我对弈数据的生成。
- 模型轻量化与部署：为了适应 Kaggle 竞赛的推理时间限制（通常为单步几秒内），可以研究模型蒸馏技术，将 AlphaZero 庞大的 ResNet 知识迁移到轻量级网络中，或优化 MCTS 的剪枝策略。
- 泛化能力扩展：目前的训练局限于标准 6×7 棋盘。未来可尝试训练支持任意尺寸棋盘的通用智能体，或将算法迁移至其他类似的棋类游戏（如五子棋、黑白棋），验证算法的通用性。
- 更复杂的博弈机制：探索基于种群的训练（Population-Based Training, PBT）来自动调整超参数，或引入 League Training 机制，构建更复杂的对手生态系统，进一步提升策略的鲁棒性。

References

[1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[2] Matteo Hessel, Joseph Modayil, Hado Van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Thirty-second AAAI Conference on Artificial Intelligence*, 2018.

[3] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

[4] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Thirtieth AAAI Conference on Artificial Intelligence*, 2016.

[5] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016.

[6] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. Dueling network architectures for deep reinforcement learning. In *International Conference on Machine Learning*, pages 1995–2003. PMLR, 2016.

[7] Meire Fortunato, Mohammad Gheshlaghi Azar, Bilal Piot, Jacob Menick, Ian Osband, Alex Graves, Vlad Mnih, Remi Munos, Demis Hassabis, Olivier Pietquin, et al. Noisy networks for exploration. In *International Conference on Learning Representations*, 2018.

[8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[9] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.